R-bloggers

R-BLOGGERS
R news and tutorials contributed by hundreds of R bloggers

HOME ABOUT RSS ADD YOUR BLOG! LEARN R R JOBS CONTACT US

Simulating dice bingo

Posted on December 30, 2021 by **kjytay** in **R bloggers** | 0 Comments

[This article was first published on **R – Statistical Odds & Ends**, and kindly contributed to R-bloggers]. (You can report issue about the content on this page here)

Want to share your content on R-bloggers? click here if you have a blog, or here if you don't.

Share Tweet

Note: This post was inspired by the "Classroom Bingo" probability puzzle in the Royal Statistical Society's *Significance* magazine (Dec 2021 edition).

Set-up

Imagine that we are playing bingo, but where the numbers are generated by the roll of two 6-sided dice with faces 1, 2, ..., 6. Each round, the two dice are rolled. If the sum of the two dice appears on your bingo card, you can strike it off. The game ends when you strike off all the numbers on your card.

For this simple game, you are asked to write just two numbers on your bingo card (which can be the same number if you wish). What two numbers should you write in order to finish the game as quickly as possible (on average)?

Intuition

With the roll of two dice, the most probable outcome is 7 (6 out of 36), followed by 6 and 8 (5 out of 36), and so on. Hence, the two numbers you should write down are probably (7, 7) or (7, 6) / (7, 8). Which is better?

Simulation

This problem space is small enough that we can estimate the number of rounds for each possible bingo card via Monte Carlo simulation. Here is the function that runs one round of dice bingo in R:

```
# Roll `nDice` dice with sides 1, 2, ..., `nFaces` and return the sum
RollDice <- function(nDice, nFaces) {
  dice <- sample(nFaces, size = nDice, replace = TRUE)
  sum(dice)
}

SimulateBingo <- function(bingoVec, nDice, nFaces, verbose = FALSE) {
  nRolls <- 0
  while (length(bingoVec) > 0) {
    nRolls <- nRolls + 1
    currentRoll <- RollDice(nDice, nFaces)

    # check if roll matches a number on card; if so, remove that number
    matchIndex <- match(currentRoll, bingoVec)
    if (!is.na(matchIndex))
      bingoVec <- bingoVec[-matchIndex]

    if (verbose) print(paste0("Roll ", nRolls, ": ", currentRoll, ", (",
                              paste(bingoVec, collapse = ","),
                              ") remaining"))
  }
  nRolls
}
```

Search R-bloggers.. Go

Your e-mail here Subscribe 52793 readers

Follow @rbloggers 96.1K followers

R bloggers Like Page

Most viewed posts (weekly)

PCA vs Autoencoders for Dimensionality Reduction
Vector Error Correction Model (VECM) using R
How to write the first for loop in R
5 Ways to Subset a Data Frame in R
Sample and Population Variance in R
ANOVA in R – How To Implement One-Way ANOVA From Scratch
How to Read Sourav Chatterjee's Basic XICOR Definition

Sponsors

The `SimulateBingo` function is flexible enough for us to have any number of numbers on our bingo card at the beginning of the game. Here's an example output from `SimulateBingo()`, where we have 3 numbers (6, 6, 7) on our card at the beginning, and at each round we roll 2 dice with sides 1 to 6:

```
set.seed(2)
SimulateBingo(c(6, 6, 7), 2, 6, verbose = TRUE)

# [1] "Roll 1: 11, (6,6,7) remaining"
# [1] "Roll 2: 7, (6,6) remaining"
# [1] "Roll 3: 6, (6) remaining"
# [1] "Roll 4: 9, (6) remaining"
# [1] "Roll 5: 3, (6) remaining"
# [1] "Roll 6: 4, (6) remaining"
# [1] "Roll 7: 9, (6) remaining"
# [1] "Roll 8: 5, (6) remaining"
# [1] "Roll 9: 7, (6) remaining"
# [1] "Roll 10: 5, (6) remaining"
# [1] "Roll 11: 9, (6) remaining"
# [1] "Roll 12: 7, (6) remaining"
# [1] "Roll 13: 11, (6) remaining"
# [1] "Roll 14: 9, (6) remaining"
# [1] "Roll 15: 6, () remaining"
# [1] 15
```

Next, we introduce a function that runs `SimulateBingo()` multiple times and returns the mean number of rounds needed to complete the game:

```
EstimateRounds <- function(nSim, bingoVec, nDice, nFaces) {
  rounds <- replicate(nSim, SimulateBingo(bingoVec, nDice, nFaces))
  mean(rounds)
}
```

(For R aficionados wondering why I didn't use `...` to pass function arguments to `SimulateBingo()`: it's because `replicate()` and `...` don't play well together. See more details here.)

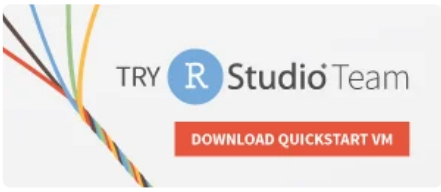
The following code runs 100,000 simulations of the game for each possible pair of numbers on the bingo card (this takes a while to run):

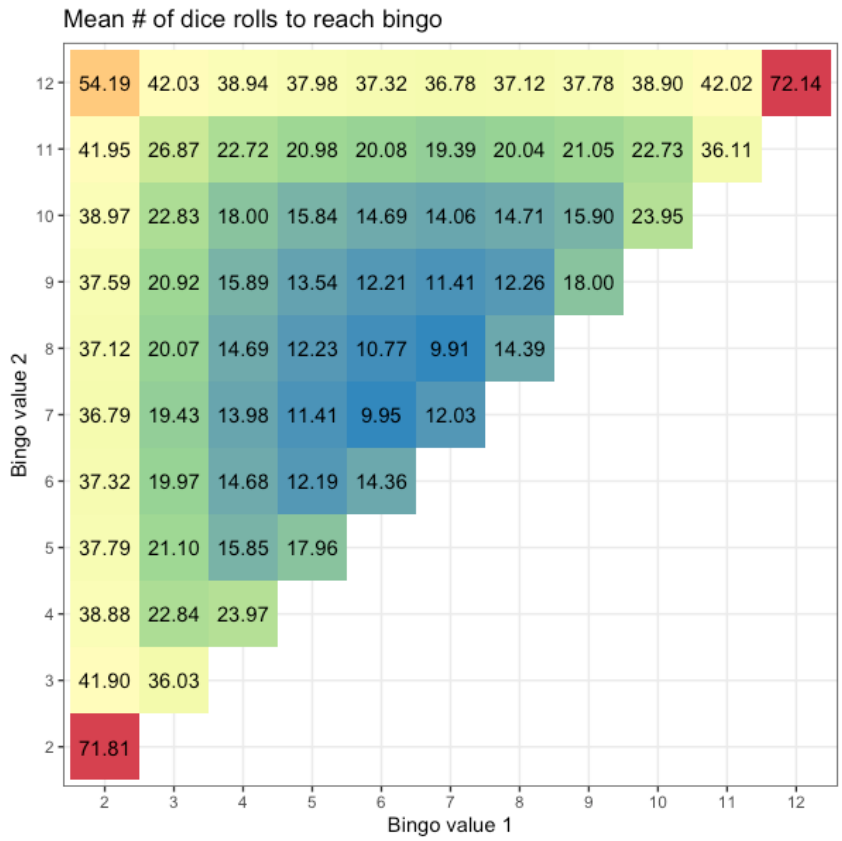
```
set.seed(2)
nDice <- 2
nFaces <- 6
nSim <- 100000

df <- data.frame(matrix(nrow = 0, ncol = 3))
maxSum <- nDice * nFaces
minSum <- nDice
for (x in minSum:maxSum) {
  for (y in x:maxSum) {
    print(paste("Simulating bingoVec = (", x, ", ", y, ")"))
    meanRounds <- EstimateRounds(nSim, bingoVec = c(x, y), nDice, nFaces)
    df <- rbind(df, c(x, y, meanRounds))
  }
}
names(df) <- c("x", "y", "mean_rounds")
```

Let's make a heatmap to visualize the results:

```
library(ggplot2)
ggplot(df, aes(factor(x), factor(y))) +
  geom_tile(aes(fill = mean_rounds)) +
  geom_text(aes(label = sprintf(mean_rounds, fmt = "%.2f"))) +
  scale_fill_distiller(palette = "Spectral") +
  labs(x = "Bingo value 1", y = "Bingo value 2",
       title = "Mean # of dice rolls to reach bingo")
  theme_update(legend.position = "null")
```





Notice the symmetry in the plot: the mean number of rounds for (x, y) is basically the same as that for $(14 - x, 14 - y)$. This makes sense due to symmetry inherent in the problem (the probability of rolling x is the same as rolling $7 - x$ for one die).

The smallest number of rounds is obtained for the pairs (6, 7) and (7, 8)!

Explanation

Let T_1 denote the number of rounds until (and including) the first time a number on the bingo card is hit, and let T_2 denote the number of rounds from then until (and including) the second number is hit. The number of rounds to complete the game is just $T = T_1 + T_2$.

Note that T_1 and T_2 are geometric random variables, and so their expectation is the reciprocal of the probability of success.

For (7, 7), the probability of success of T_1 and T_2 is the same: it's the probability of rolling a 7, which is $6/36 = 1/6$. Hence,

$$\mathbb{E}[T] = \mathbb{E}[T_1] + \mathbb{E}[T_2] = 6 + 6 = 12.$$

For (6, 7), the computation is a bit trickier because the distribution of T_2 depends on whether the 6 or the 7 was hit first. If 6 was hit first, then $T_2 \sim \text{Geom}(6/36)$, and if 7 was hit first, then $T_2 \sim \text{Geom}(5, 36)$. What is the probability that 6 is hit first? It's

$$\text{Prob 6 first} = \frac{5/36}{5/36 + 6/36} = \frac{5}{11}.$$

Hence,

$$\begin{aligned}\mathbb{E}[T_2] &= \mathbb{E}[\text{Geom}(6/36)] \cdot \frac{5}{11} + \mathbb{E}[\text{Geom}(5/36)] \cdot \left(1 - \frac{5}{11}\right) \\ &= \frac{366}{55} \\ &\approx 6.65\end{aligned}$$

T_1 is a geometric distribution with success probability equal to the probability that 6 or 7 is rolled, i.e. $5/36 + 6/36 = 11/36$, implying that $\mathbb{E}[T_1] = 36/11$. Hence,

Learn R (for ecology)

DATAKOLEKTIV
In-depth R training
>_

Our ads respect your privacy. Read our Privacy Policy page to learn more.

Contact us if you wish to help support R-bloggers, and place **your banner here**.

Recent Posts

Simulating dice bingo
Get and Set working directory (setwd / getwd) in R
R User Group Kano, Nigeria, is Bringing in the Next Generation of Students to the R Community
Estimating recent population history from linkage disequilibrium with GONE and SNeP
Exploring the XI Correlation Coefficient
Examining College Football Conference Realignment with {ggraph}
Gold-Mining Week 17 (2021)
Some Interesting Issues in VECM using R
Sample and Population Variance in R
triple ruin
How the Lagos R User Group Has Leveraged the Pandemic
Exploring College Football Non-Conference Rivalries with {ggraph}
How to Read Sourav Chatterjee's Basic XICOR Definition
Advent of 2021, Day 25 – Spark literature, documentation, courses and books
Vector Error Correction Model (VECM) using R

Jobs for R-users

Junior Data Scientist / Quantitative economist
Senior Quantitative Analyst
R programmer
Data Scientist – CGIAR Excellence in Agronomy (Ref No: DDG-R4D/DS/1/CG/EA/06/20)
Data Analytics Auditor, Future of Audit Lead @ London or Newcastle

python-bloggers.com (python/data-science news)

Trigonometric Functions in Python: Complete Guide
Online workshop: Analytics from Excel to Python and R

$$\begin{aligned}\mathbb{E}[T] &= \mathbb{E}[T_1] + \mathbb{E}[T_2] \\ &= \frac{36}{11} + \frac{366}{55} \\ &= \frac{546}{55} \\ &\approx 9.93,\end{aligned}$$

which is quite a bit less than the expected value for (7, 7)! (Notice that these values are very close to the simulated results.)

On second thought this makes sense: even though it takes longer to hit the second number in (6, 7), this is more than made up for in the time it takes to hit the first number, since we can hit either 6 or 7, not just 7. (Looking at the simulation results suggests that quite a few other pairs of numbers are better than (7, 7).)

Extensions

There are many interesting questions to ponder as extensions to this game:

1. What if you had to write down 3 numbers on your bingo card instead? Or 4? Or *n*?
2. What if there were more dice?
3. What if the dice had non-standard faces?

Related

Probability of Getting Zero Points in Dr. Burnörium's Psycho Dice Game
"What is the probability of ending a round with zero points in Dr. Burnörium's Psycho Dice Russian Roulette Game?" That
October 18, 2018
In "R bloggers"

How many rounds of Tenzi does it take to win?
I spent some time over Christmas playing games of chance with my in-laws. Some of them didn't have any strategy at all, and
January 29, 2019
In "R bloggers"

Simulating simple dice games by @ellis2013nz
A necessary but not sufficient condition for a game being one of skill rather than pure chance is that the player gets to
October 26, 2018
In "R bloggers"

Share Tweet

To **leave a comment** for the author, please follow the link and comment on their blog: **R – Statistical Odds & Ends**.

R-bloggers.com offers **daily e-mail updates** about R news and tutorials about learning R and many other topics. Click here if you're looking to post or find an R/data-science job.

Want to share your content on R-bloggers? click here if you have a blog, or here if you don't.

← **Previous post**

Copyright © 2021 | MH Corporate basic by MH Themes

Thinking about maps and ice cream
Job vacancies at Jumping Rivers!
How to Create a Simple Streamlit App + How to Deploy it on Heroku
Jaccard similarity and Jaccard distance in Python
Top 7 Skills to Get Hired in Data Science in 2022

Full list of contributing R-bloggers

Archives

Select Month

Other sites

SAS blogs
Jobs for R-users